PSet 1: The Price of Housing

```
# Setup and data import
library(tidyverse)

DATA_FILE <- "fmr2027.csv"  # relative to this .qmd file
REF_BEDROOMS <- 2           # bedroom count for single-size views and ratios
N_EXTREME <- 12             # areas shown at each end of the extremes plot
JITTER_SEED <- 1            # deterministic jitter
```

```
fmr_raw <- read_csv(
  DATA_FILE,
  col_types = cols(
    fips = col_character(), stusps = col_character(),
    state = col_character(), countyname = col_character(),
    county_town_name = col_character(), hud_area_code = col_character(),
    hud_area_name = col_character(), metro = col_integer(),
    pop2024 = col_double(), bedrooms = col_integer(), fmr = col_double()
  )
)

# fail loudly if the structure we rely on is not there
stopifnot(
  nrow(problems(fmr_raw)) == 0,
  all(fmr_raw$metro %in% c(0L, 1L)),
  REF_BEDROOMS %in% fmr_raw$bedrooms
)
```

```
# Records: one row per geographic record and bedroom count.
fmr <- fmr_raw |>
  mutate(
    metro = factor(metro, levels = c(0, 1), labels = c("nonmetro", "metro")),
    # fips = state (2) + county (3) + county subdivision (5);
    # a 99999 subdivision code means the whole county
    record_type = if_else(str_ends(fips, "99999"), "county", "sub-county")
  )

# Areas: one row per HUD FMR area and bedroom count. FMR must be identical
# for every record in an area; population is summed over the records.
stopifnot(
  fmr |>
    summarise(k = n_distinct(fmr), .by = c(hud_area_code, bedrooms)) |>
    pull(k) |>
    (\(k) all(k == 1))()
)

areas <- fmr |>
  summarise(
    fmr = first(fmr),
    pop = sum(pop2024),
    n_records = n(),
    .by = c(hud_area_code, hud_area_name, metro, bedrooms)
```

```
  )

# An area that crosses a state line is kept once in each state it touches.
area_states <- fmr |>
  distinct(hud_area_code, stusps, metro, bedrooms, fmr)

bedroom_levels <- sort(unique(fmr$bedrooms))
records_ref <- filter(fmr, bedrooms == REF_BEDROOMS)
areas_ref <- filter(areas, bedrooms == REF_BEDROOMS)
```

```
state_counts <- fmr |>
  filter(bedrooms == REF_BEDROOMS) |>
  summarise(
    records = n(),
    `HUD areas` = n_distinct(hud_area_code),
    .by = stusps
  ) |>
  pivot_longer(-stusps, names_to = "unit", values_to = "count") |>
  mutate(stusps = fct_reorder(stusps, count, .fun = max))

ggplot(state_counts, aes(count, stusps)) +
  geom_line(aes(group = stusps), colour = "grey70") +
  geom_point(aes(colour = unit), size = 2) +
  scale_x_log10(breaks = c(1, 3, 10, 30, 100, 300)) +
  labs(x = "count", y = "stusps", colour = "unit", caption = "Figure 1") +
  theme_minimal(12)
```

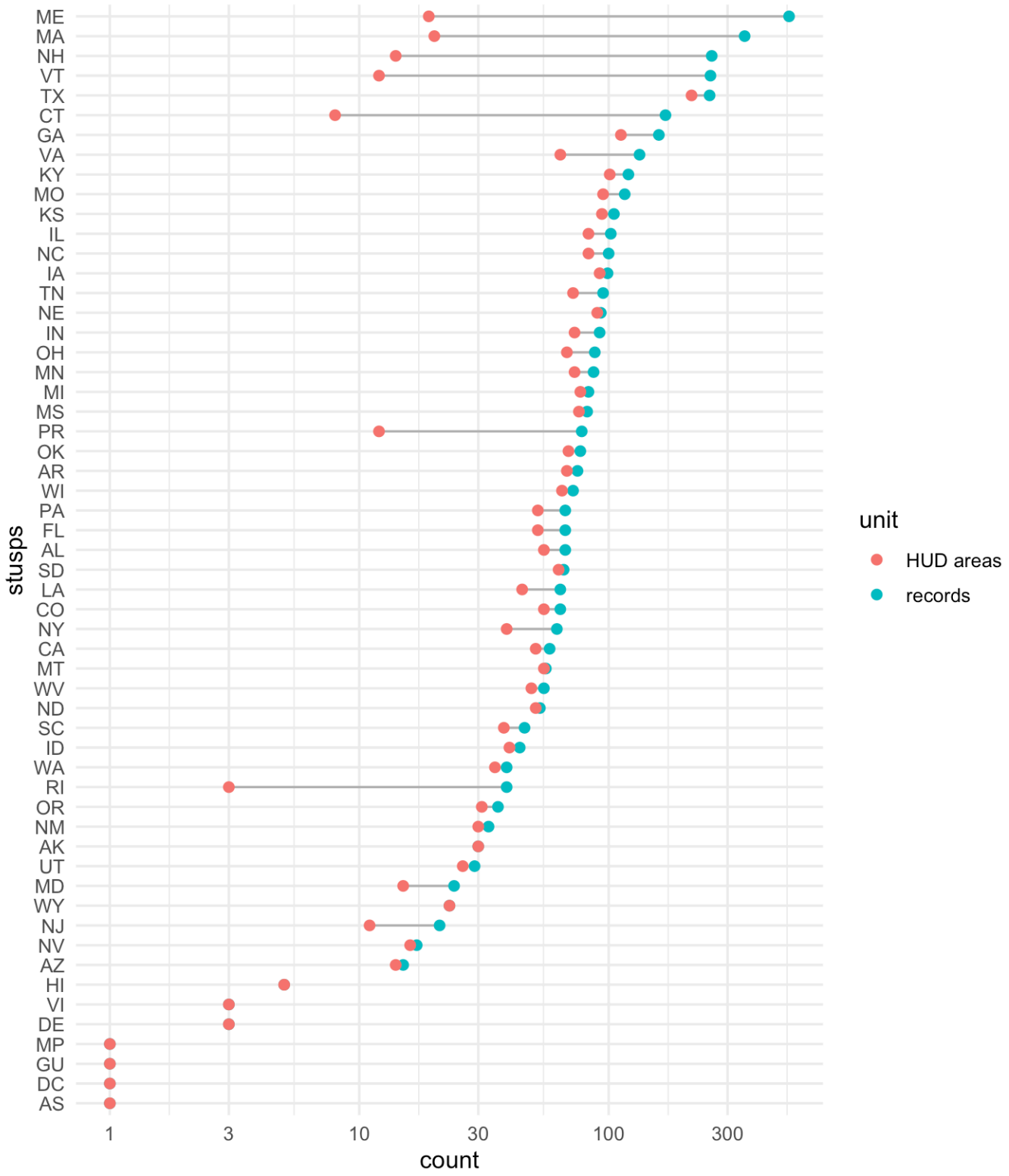


Figure 1

```r
ggplot(areas, aes(fmr)) +
  geom_histogram(binwidth = 100, boundary = 0) +
  facet_grid(rows = vars(bedrooms), labeller = label_both) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(x = "fmr ($)", y = "count (HUD areas)", caption = "Figure 2") +
  theme_minimal(12)
```

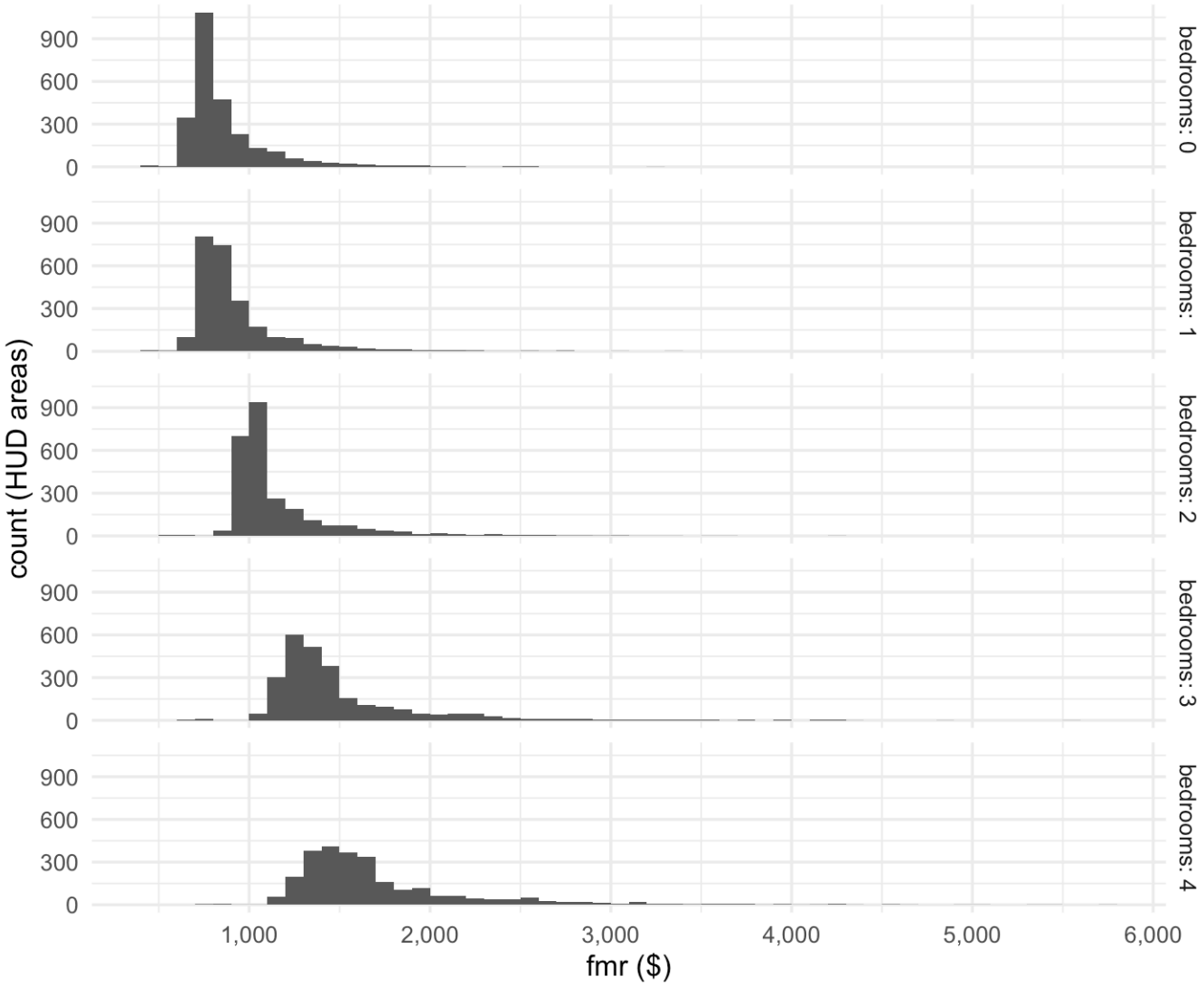


Figure 2

```
bedroom_ratios <- areas |>
  select(hud_area_code, bedrooms, fmr) |>
  mutate(ratio = fmr / fmr[bedrooms == REF_BEDROOMS], .by = hud_area_code) |>
  filter(bedrooms != REF_BEDROOMS)

ggplot(bedroom_ratios, aes(ratio)) +
  geom_histogram(binwidth = 0.01, boundary = 0) +
  facet_wrap(vars(bedrooms), scales = "free_x", labeller = label_both) +
  scale_x_continuous(breaks = scales::breaks_pretty(n = 4)) +
  labs(
    x = paste0("fmr / fmr (", REF_BEDROOMS, " bedrooms)"),
    y = "count (HUD areas)",
    caption = "Figure 3"
  ) +
  theme_minimal(12)
```

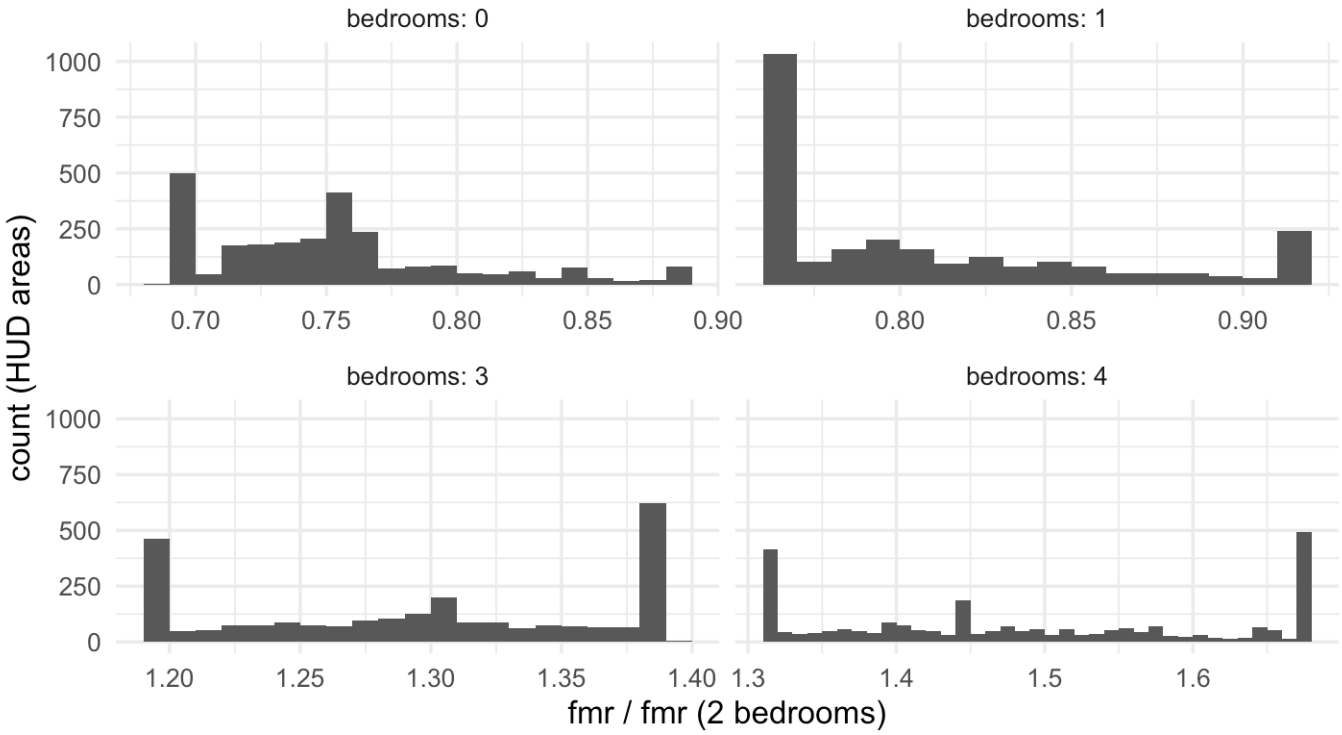


Figure 3

```
state_areas <- area_states |>
  filter(bedrooms == REF_BEDROOMS) |>
  mutate(stusps = fct_reorder(stusps, fmr, .fun = median))

ggplot(state_areas, aes(fmr, stusps)) +
  geom_point(
    aes(colour = metro), alpha = 0.6, size = 1.2,
    position = position_jitter(width = 0, height = 0.25, seed = JITTER_SEED)
  ) +
  stat_summary(fun = median, geom = "point", shape = 124, size = 4) +
  facet_wrap(vars(bedrooms), labeller = label_both) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(x = "fmr ($)", y = "stusps", colour = "metro", caption = "Figure 4") +
  theme_minimal(12)
```

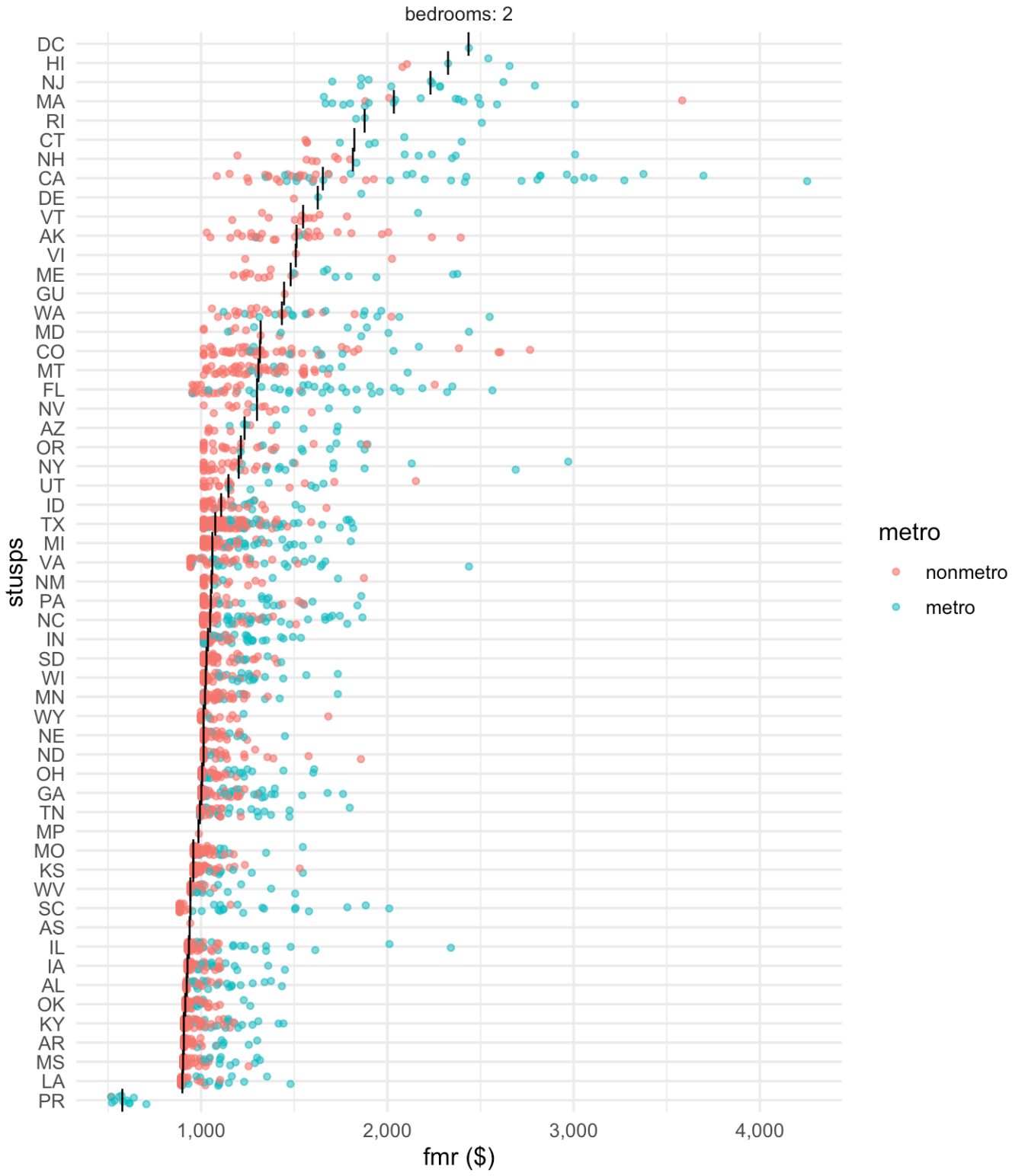


Figure 4

```r
ggplot(areas, aes(factor(bedrooms), fmr, fill = metro)) +
  geom_boxplot(outlier.size = 0.8, outlier.alpha = 0.4) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(x = "bedrooms", y = "fmr ($)", fill = "metro", caption = "Figure 5") +
  theme_minimal(12)
```

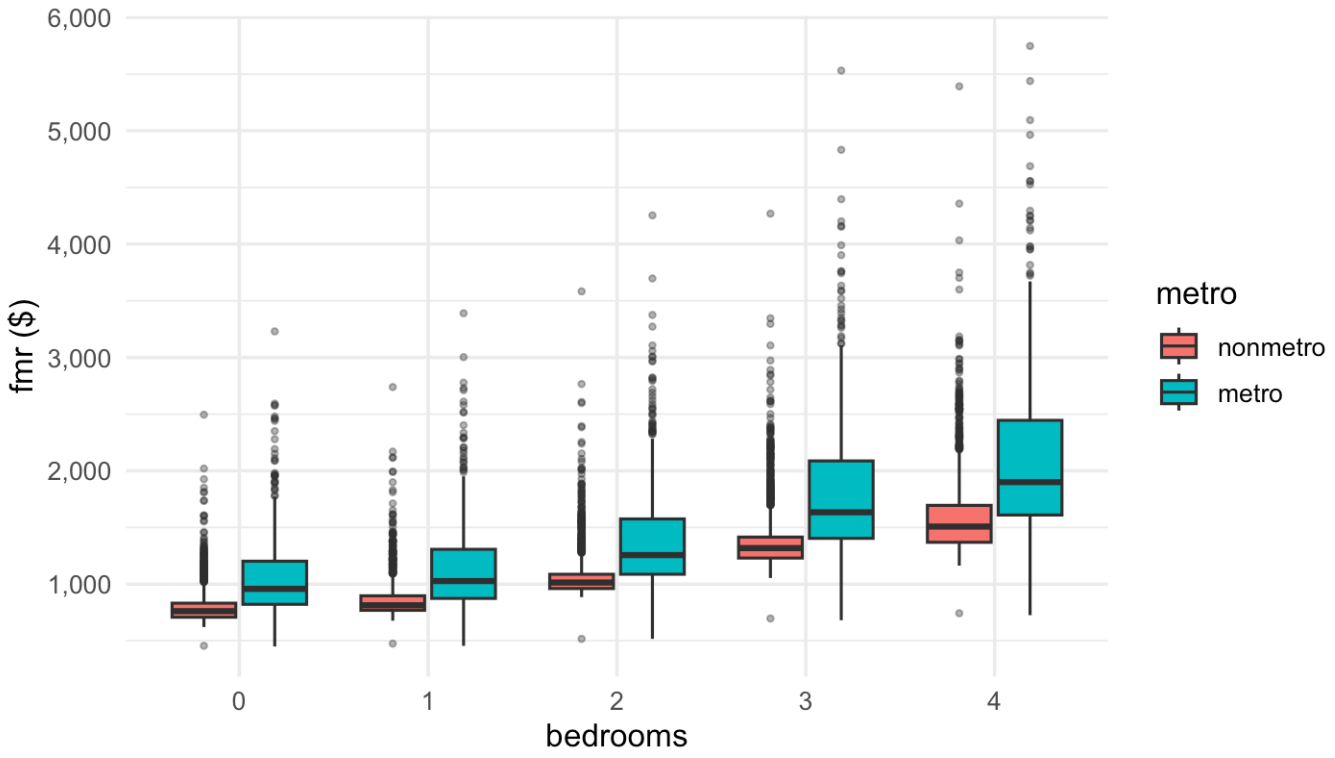


Figure 5

```
ggplot(filter(areas, bedrooms == REF_BEDROOMS), aes(pop, fmr)) +
  geom_point(aes(colour = metro), alpha = 0.4, size = 1) +
  facet_wrap(vars(bedrooms), labeller = label_both) +
  scale_x_log10(labels = scales::label_comma()) +
  scale_y_continuous(labels = scales::label_comma()) +
  labs(
    x = "pop2024 (area sum)", y = "fmr ($)", colour = "metro",
    caption = "Figure 6"
  ) +
  theme_minimal(12)
```

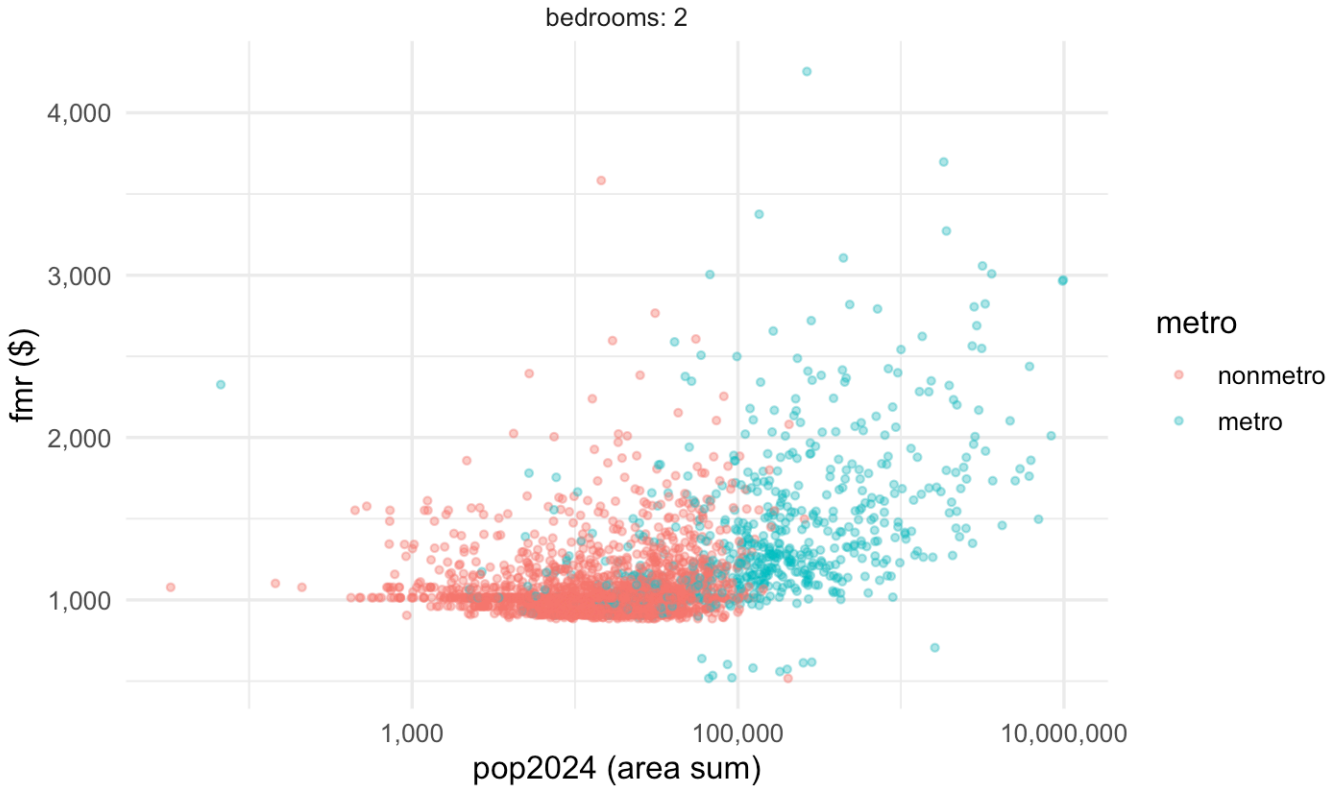


Figure 6

```
# pseudo-log axis: linear near zero so zero-population records stay visible
ggplot(filter(fmr, bedrooms == REF_BEDROOMS), aes(pop2024)) +
  geom_histogram(bins = 40) +
  facet_grid(rows = vars(record_type), labeller = label_both) +
  scale_x_continuous(
    transform = scales::transform_pseudo_log(),
    breaks = c(0, 10^(1:7)),
    labels = scales::label_comma()
  ) +
  labs(x = "pop2024", y = "count (records)", caption = "Figure 7") +
  theme_minimal(12)
```

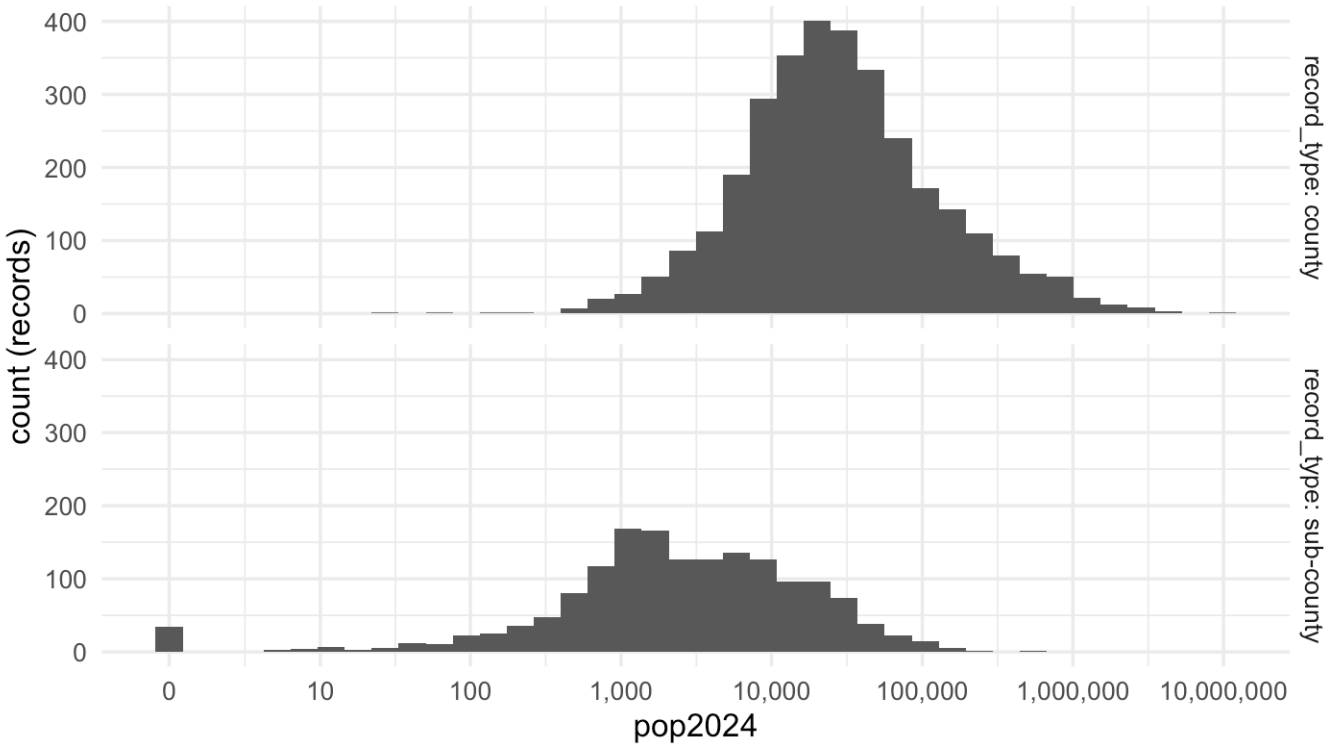


Figure 7

```
weighted_ecdf <- function(fmr, weight, unit) {
  o <- order(fmr)
  weight <- rep_len(weight, length(fmr))
  tibble(
    bedrooms = REF_BEDROOMS,
    fmr = fmr[o],
    cumulative = cumsum(weight[o]) / sum(weight),
    unit = unit
  )
}

ecdf_units <- bind_rows(
  weighted_ecdf(records_ref$fmr, 1, "record"),
  weighted_ecdf(areas_ref$fmr, 1, "HUD area"),
  weighted_ecdf(records_ref$fmr, records_ref$pop2024, "population")
)

ggplot(ecdf_units, aes(fmr, cumulative, colour = unit)) +
  geom_step() +
  facet_wrap(vars(bedrooms), labeller = label_both) +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(
    x = "fmr ($)", y = "cumulative proportion", colour = "unit",
    caption = "Figure 8"
  ) +
  theme_minimal(12)
```

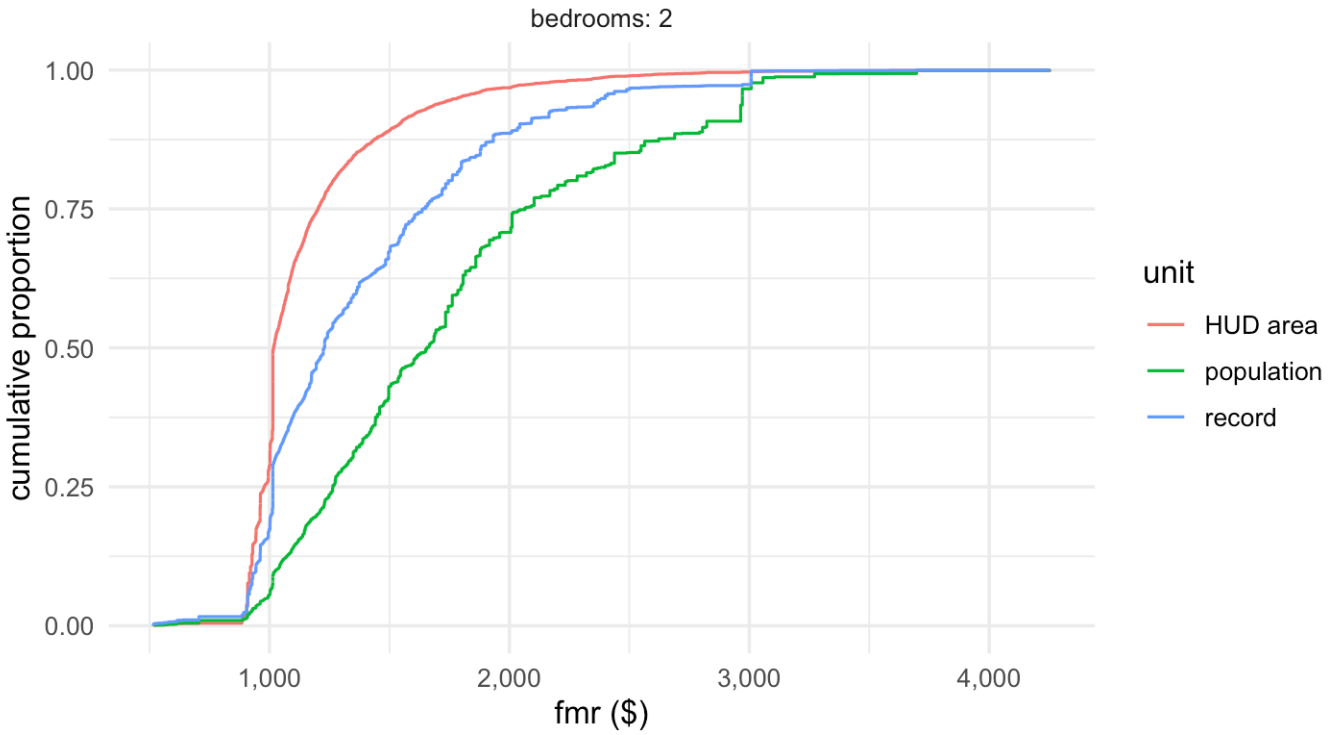


Figure 8

```
ranked_areas <- areas |>
  filter(bedrooms == REF_BEDROOMS) |>
  arrange(fmr, hud_area_name)

extreme_codes <- c(
  head(ranked_areas$hud_area_code, N_EXTREME),
  tail(ranked_areas$hud_area_code, N_EXTREME)
)

extreme_areas <- areas |>
  filter(hud_area_code %in% extreme_codes) |>
  mutate(
    hud_area_name = str_remove(hud_area_name, " HUD Metro FMR Area"),
    hud_area_name = fct_reorder(
      hud_area_name, if_else(bedrooms == REF_BEDROOMS, fmr, NA_real_),
      .fun = max, na.rm = TRUE
    ),
    bedrooms = factor(bedrooms)
  )

ggplot(extreme_areas, aes(fmr, hud_area_name, colour = bedrooms)) +
  geom_point(size = 2) +
  scale_colour_viridis_d() +
  scale_x_continuous(labels = scales::label_comma()) +
  labs(
    x = "fmr ($)", y = "hud_area_name", colour = "bedrooms",
    caption = "Figure 9"
  ) +
  theme_minimal(12)
```

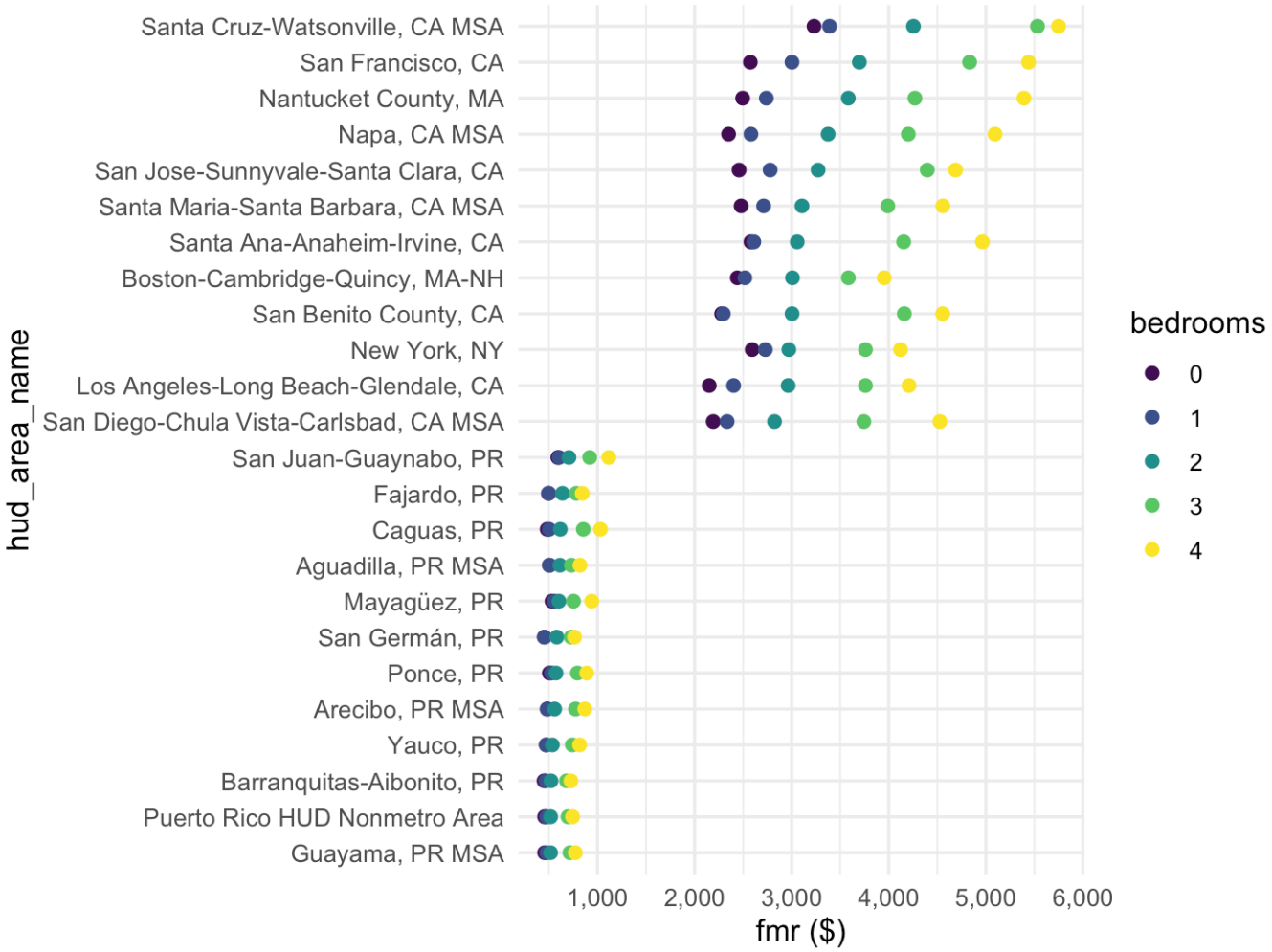

Santa Cruz-Watsonville, CA MSA
San Francisco, CA
Nantucket County, MA
Napa, CA MSA
San Jose-Sunnyvale-Santa Clara, CA
Santa Maria-Santa Barbara, CA MSA
Santa Ana-Anaheim-Irvine, CA
Boston-Cambridge-Quincy, MA-NH
San Benito County, CA
New York, NY
Los Angeles-Long Beach-Glendale, CA
San Diego-Chula Vista-Carlsbad, CA MSA
San Juan-Guaynabo, PR
Fajardo, PR
Caguas, PR
Aguadilla, PR MSA
Mayagüez, PR
San Germán, PR
Ponce, PR
Arecibo, PR MSA
Yauco, PR
Barranquitas-Aibonito, PR
Puerto Rico HUD Nonmetro Area
Guayama, PR MSA
hud_area_name
1,000
2,000
3,000
4,000
5,000
6,000
fmr ($)
bedrooms
0
1
2
3
4

Figure 9

Supporting exploration and data checks

Figures 1–9 above are the final graphs. The tables below document the exploration and data checks behind them, using `fmr2027.csv`. Each check states its unit of analysis: *record* (one geographic record), *area* (one HUD FMR area), or *population-weighted*.

Dataset structure

```
cat(
  "rows:", nrow(fmr_raw), " columns:", ncol(fmr_raw),
  "\ngeographic records (distinct fips):", n_distinct(fmr_raw$fips),
  "\nHUD FMR areas (distinct hud_area_code):",
  n_distinct(fmr_raw$hud_area_code),
  "\nbedroom values:", paste(bedroom_levels, collapse = ", "), "\n"
)
```

```
rows: 23820  columns: 11
geographic records (distinct fips): 4764
HUD FMR areas (distinct hud_area_code): 2606
bedroom values: 0, 1, 2, 3, 4
```

```
fmr_raw |>
  summarise(across(
    everything(),
    list(
      type = \(x) class(x)[1],
      unique = n_distinct,
      missing = \(x) sum(is.na(x)),
      min = \(x) if (is.numeric(x)) min(x, na.rm = TRUE) else NA,
      max = \(x) if (is.numeric(x)) max(x, na.rm = TRUE) else NA
    ),
    .names = "{.col}__{.fn}"
  )) |>
  pivot_longer(
    everything(), names_to = c("column", ".value"), names_sep = "__"
  ) |>
  mutate(
    pct_missing = round(100 * missing / nrow(fmr_raw), 2),
    .after = missing
  ) |>
  knitr::kable()
```

column	type	unique	missing	pct_missing	min	max
fips	character	4764	0	0.00	NA	NA
stusps	character	56	0	0.00	NA	NA
state	character	56	0	0.00	NA	NA
countyname	character	1966	0	0.00	NA	NA
county_town_name	character	1255	15795	66.31	NA	NA
hud_area_code	character	2606	0	0.00	NA	NA
hud_area_name	character	2606	0	0.00	NA	NA
metro	integer	2	0	0.00	0	1
pop2024	numeric	4467	0	0.00	0	9808667

column	type	unique	missing	pct_missing	min	max
bedrooms	integer	5	0	0.00	0	4
fmr	numeric	1965	0	0.00	450	5748

Duplicates, keys, and completeness

```
tibble(
  check = c(
    "fully duplicated rows",
    "duplicated (fips, bedrooms) keys",
    "fips values that do not have every bedroom count",
    "fips with more than one countyname, pop2024, metro, or area",
    "fips whose first two digits differ from state",
    "state codes that map to more than one stusps"
  ),
  count = c(
    sum(duplicated(fmr_raw)),
    sum(duplicated(fmr_raw[c("fips", "bedrooms")])),
    sum(count(fmr_raw, fips)$n != length(bedroom_levels)),
    fmr_raw |>
      summarise(
        bad = n_distinct(countyname) > 1 | n_distinct(pop2024) > 1 |
          n_distinct(metro) > 1 | n_distinct(hud_area_code) > 1,
        .by = fips
      ) |>
      pull(bad) |>
      sum(),
    sum(str_sub(fmr_raw$fips, 1, 2) != fmr_raw$state),
    sum(count(distinct(fmr_raw, state, stusps), state)$n > 1)
  )
) |>
  knitr::kable()
```

check	count
fully duplicated rows	0
duplicated (fips, bedrooms) keys	0
fips values that do not have every bedroom count	0
fips with more than one countyname, pop2024, metro, or area	0
fips whose first two digits differ from state	0
state codes that map to more than one stusps	0

Missing town names and record types

```
fmr |>
  filter(bedrooms == REF_BEDROOMS) |>
  summarise(
    records = n(),
    missing_town_name = sum(is.na(county_town_name)),
    .by = record_type
  ) |>
  knitr::kable()
```

record_type	records	missing_town_name
county	3161	3159
sub-county	1603	0

```
fmr |>
  filter(bedrooms == REF_BEDROOMS, record_type == "sub-county") |>
  count(stusps, name = "sub_county_records") |>
  knitr::kable()
```

stusps	sub_county_records
CT	169
MA	351
ME	529
NH	259
RI	39
VT	256

```
# county-level records that nevertheless carry a name
fmr |>
  filter(
    bedrooms == REF_BEDROOMS, record_type == "county",
    !is.na(county_town_name)
  ) |>
  select(fips, stusps, countyname, county_town_name, pop2024) |>
  knitr::kable()
```

fips	stusps	countyname	county_town_name	pop2024
0219599999	AK	Petersburg Borough	Petersburg Census Area	3409
2905699999	MO	Sullivan part	Sullivan city part of Crawford County	927

Repeated HUD areas and rent schedules

```
multi_state_areas <- fmr |>
  filter(bedrooms == REF_BEDROOMS) |>
  summarise(n_states = n_distinct(stusps), .by = hud_area_code) |>
  filter(n_states > 1)

areas_ref |>
  summarise(
    areas = n(),
    areas_with_1_record = sum(n_records == 1),
    areas_with_2plus_records = sum(n_records > 1),
    records_in_shared_areas = sum(n_records[n_records > 1]),
    max_records_in_one_area = max(n_records),
    areas_spanning_states = nrow(multi_state_areas)
  ) |>
  pivot_longer(everything(), names_to = "measure", values_to = "value") |>
  knitr::kable()
```

measure	value
areas	2606
areas_with_1_record	2325
areas_with_2plus_records	281
records_in_shared_areas	2439
max_records_in_one_area	114
areas_spanning_states	36

```
areas_ref |>
  arrange(desc(n_records)) |>
  select(hud_area_name, n_records, pop, fmr) |>
  head(5) |>
  knitr::kable()
```

hud_area_name	n_records	pop	fmr
Boston-Cambridge-Quincy, MA-NH HUD Metro FMR Area	114	3605178	3008
Aroostook County, ME	71	67058	1175
Hartford-West Hartford-East Hartford, CT MSA	55	1153112	1933
Penobscot County, ME (part) HUD Metro FMR Area	53	58215	1496
Washington County, ME	48	31331	1230

```
# different areas can also share an identical set of rents
areas |>
  pivot_wider(
    id_cols = hud_area_code, names_from = bedrooms, values_from = fmr,
    names_prefix = "bed"
  ) |>
  summarise(
    areas = n(),
    distinct_schedules = nrow(distinct(pick(starts_with("bed"))))
  ) |>
  knitr::kable()
```

areas	distinct_schedules
2606	2446

Metropolitan coding

```
fmr |>
  filter(bedrooms == REF_BEDROOMS) |>
  mutate(area_prefix = str_extract(hud_area_code, "^[A-Z]+")) |>
  count(metro, area_prefix, name = "records") |>
  knitr::kable()
```

metro	area_prefix	records
nonmetro	NCNTY	2788
metro	METRO	1976

```
areas_ref |>
  summarise(
    areas = n(),
    records = sum(n_records),
    population = sum(pop),
    .by = metro
  ) |>
  knitr::kable()
```

metro	areas	records	population
metro	649	1976	291976636
nonmetro	1957	2788	46557544

Population values

```
fmr |>
  filter(bedrooms == REF_BEDROOMS) |>
  summarise(
    records = n(),
    zero_pop = sum(pop2024 == 0),
    below_100 = sum(pop2024 > 0 & pop2024 < 100),
    median_pop = median(pop2024),
    max_pop = max(pop2024),
    .by = record_type
  ) |>
  knitr::kable()
```

record_type	records	zero_pop	below_100	median_pop	max_pop
county	3161	0	2	25483	9808667
sub-county	1603	34	57	2622	666442

```
fmr |>
  filter(bedrooms == REF_BEDROOMS, pop2024 == 0) |>
  count(stusps, metro, name = "zero_pop_records") |>
  knitr::kable()
```

stusps	metro	zero_pop_records
ME	nonmetro	8
ME	metro	3
NH	nonmetro	17
VT	nonmetro	6

```
fmr |>
  filter(bedrooms == REF_BEDROOMS, pop2024 == 0) |>
  select(stusps, countyname, county_town_name) |>
  head(8) |>
  knitr::kable()
```

stusps	countyname	county_town_name
ME	Aroostook County	Penobscot Indian Island Reservation
ME	Franklin County	West Central Franklin UT
ME	Hancock County	Marshall Island UT
ME	Hancock County	Northwest Hancock UT
ME	Knox County	Criehaven UT
ME	Knox County	Muscle Ridge Island UT
ME	Lincoln County	Hibberts gore
ME	Lincoln County	Louds Island UT

```
cat("areas with zero total population:", sum(areas_ref$pop == 0), "\n")
```

```
areas with zero total population: 0
```

Distribution of fmr and bedroom structure

```
quartile_summary <- function(data, label) {
  data |>
    summarise(
      n = n(), min = min(fmr),
      q1 = quantile(fmr, 0.25), median = median(fmr),
      q3 = quantile(fmr, 0.75), max = max(fmr), mean = mean(fmr),
      # Bowley skewness: > 0 means a longer upper half of the box
      bowley = ((q3 - median) - (median - q1)) / (q3 - q1),
      .by = bedrooms
    ) |>
    mutate(unit = label, .before = 1) |>
    arrange(bedrooms)
}

bind_rows(
  quartile_summary(fmr, "record"),
  quartile_summary(areas, "area")
) |>
  mutate(across(where(is.double), \(x) round(x, 2))) |>
  knitr::kable()
```

unit	bedrooms	n	min	q1	median	q3	max	mean	bowley
record	0	4764	450	764.00	933.0	1242.00	3230	1049.93	0.29
record	1	4764	455	819.00	977.5	1329.00	3390	1124.52	0.38
record	2	4764	517	1014.00	1228.0	1642.00	4254	1387.64	0.32
record	3	4764	681	1326.00	1611.0	2134.25	5531	1784.25	0.29
record	4	4764	726	1514.00	1849.5	2445.00	5748	2028.48	0.28
area	0	2606	450	718.25	783.0	915.75	3230	866.74	0.34
area	1	2606	455	775.00	841.0	970.00	3390	930.06	0.32
area	2	2606	517	979.00	1017.0	1202.00	4254	1149.78	0.66

unit	bedrooms	n	min	q1	median	q3	max	mean	bowley
area	3	2606	681	1252.00	1368.0	1547.00	5531	1490.68	0.21
area	4	2606	726	1402.00	1562.5	1832.00	5748	1713.92	0.25

```
# exactly repeated two-bedroom values across HUD areas
areas_ref |>
  summarise(
    areas = n(),
    states = n_distinct(area_states$stusps[
      area_states$hud_area_code %in% hud_area_code
    ]),
    nonmetro_areas = sum(metro == "nonmetro"),
    .by = fmr
  ) |>
  arrange(desc(areas)) |>
  mutate(share_of_areas = round(areas / nrow(areas_ref), 3)) |>
  head(5) |>
  knitr::kable()
```

fmr	areas	states	nonmetro_areas	share_of_areas
1014	372	20	351	0.143
909	98	2	93	0.038
1002	67	2	62	0.026
961	66	2	64	0.025
962	62	5	58	0.024

```
# monotonicity within each geographic record
fmr |>
  arrange(fips, bedrooms) |>
  summarise(
    non_decreasing = all(diff(fmr) >= 0),
    strictly_increasing = all(diff(fmr) > 0),
    .by = fips
  ) |>
  summarise(
    records = n(),
    non_decreasing = sum(non_decreasing),
    strictly_increasing = sum(strictly_increasing)
  ) |>
  knitr::kable()
```

records	non_decreasing	strictly_increasing
4764	4764	4756

```
bedroom_ratios |>
  summarise(
    min = min(ratio), median = median(ratio), max = max(ratio),
    areas_at_min_ratio = sum(ratio < min(ratio) + 0.005),
    areas_at_max_ratio = sum(ratio > max(ratio) - 0.005),
```

```
    n_areas = n(),
    .by = bedrooms
  ) |>
  mutate(across(where(is.double), \(x) round(x, 3))) |>
  knitr::kable()
```

bedrooms	min	median	max	areas_at_min_ratio	areas_at_max_ratio	n_areas
0	0.689	0.750	0.882	6	82	2606
1	0.762	0.791	0.919	994	228	2606
3	1.191	1.304	1.391	439	6	2606
4	1.314	1.474	1.679	411	10	2606

Geographic comparisons: the unit of analysis matters

```
weighted_median <- function(x, w) {
  if (sum(w) == 0) return(NA_real_)
  o <- order(x)
  x[o][which(cumsum(w[o]) >= sum(w) / 2)[1]]
}

state_summary <- full_join(
  fmr |>
    filter(bedrooms == REF_BEDROOMS) |>
    summarise(
      records = n(),
      median_record = median(fmr),
      median_pop_weighted = weighted_median(fmr, pop2024),
      .by = stusps
    ),
  state_areas |>
    summarise(areas = n(), median_area = median(fmr), .by = stusps),
  by = "stusps"
)

state_summary |>
  mutate(gap = abs(median_record - median_area)) |>
  arrange(desc(gap)) |>
  head(10) |>
  mutate(across(where(is.double), \(x) round(x))) |>
  knitr::kable()
```

stusps	records	median_record	median_pop_weighted	areas	median_area	gap
MD	24	1875	2006	15	1320	555
MA	351	2367	3008	20	2038	328
VA	133	1262	1667	64	1060	202
NY	62	1385	2689	39	1205	180
FL	67	1474	1917	52	1304	170
SC	46	1098	1329	38	944	154
ME	529	1367	1677	19	1484	117

stusps	records	median_record	median_pop_weighted	areas	median_area	gap
OR	36	1324	1878	31	1214	110
CT	169	1933	1933	8	1824	110
NC	100	1145	1386	83	1050	95

```
# share of between-area variation in log(fmr) associated with state
cat(
  "R-squared, log(fmr) ~ state (one row per area-state pair):",
  round(summary(lm(log(fmr) ~ stusps, data = state_areas))$r.squared, 3), "\n"
)
```

```
R-squared, log(fmr) ~ state (one row per area-state pair): 0.572
```

```
areas_ref |>
  summarise(
    areas = n(),
    q1 = quantile(fmr, 0.25), median = median(fmr), q3 = quantile(fmr, 0.75),
    .by = metro
  ) |>
  knitr::kable()
```

metro	areas	q1	median	q3
metro	649	1087	1256	1575
nonmetro	1957	961	1014	1087

Population and fmr

```
# Spearman: rank-based, so it is not distorted by the skewed population scale
bind_rows(
  fmr |>
    filter(bedrooms == REF_BEDROOMS) |>
    summarise(
      unit = "record", n = n(),
      rho = cor(pop2024, fmr, method = "spearman"),
      .by = record_type
    ) |>
    rename(group = record_type),
  areas_ref |>
    summarise(
      unit = "area", n = n(),
      rho = cor(pop, fmr, method = "spearman"),
      .by = metro
    ) |>
    mutate(group = as.character(metro), .keep = "unused")
) |>
  mutate(rho = round(rho, 3)) |>
  knitr::kable()
```

group	unit	n	rho
county	record	3161	0.458

group	unit	n	rho
sub-county	record	1603	0.714
metro	area	649	0.576
nonmetro	area	1957	0.106

Reproducibility

```
tibble(
  component = c("R", "tidyverse", "ggplot2", "dplyr", "readr", "scales"),
  version = c(
    as.character(getRversion()),
    map_chr(
      c("tidyverse", "ggplot2", "dplyr", "readr", "scales"),
      \(p) as.character(packageVersion(p))
    )
  )
) |>
  knitr::kable()
```

component	version
R	4.6.1
tidyverse	2.0.0
ggplot2	4.0.3
dplyr	1.2.1
readr	2.2.0
scales	1.4.0